

Assignment 2

Key Information

Academic Integrity

Your code will be checked against all other students with a similarity checker.

Anything you are able to google can be easily found by the teaching team and compared against your work.

In this assessment, you must not use generative artificial intelligence (genAI) to generate any materials or content in relation to the assessment task. This includes ChatGPT and GitHub Copilot.

You are responsible for all code submitted as a part of your assignment.

How can I avoid academic integrity issues?

- Never copy code from anywhere. If you learn something useful online, rewrite it from scratch. It's also the best way to make sure you have understood it. This includes not submitting code you have submitted in a prior assessment in this or another unit. Self-plagiarism is also a breach of academic integrity.
- If a fellow student asks you for a solution to a question, you can try to help them build their solution, but **do not give them yours**. Giving your solution is just as much of an academic Integrity breach as receiving it.
- Don't leave your devices unlocked and unattended. Properly securing your work is your responsibility.

Assignment design

There are 3 independent parts to this assignment.

Each part is designed to cover 1 week of content and take one week of group work to complete.

At the end of every week but the last, there is a "checkpoint", at which point we expect you to have completed the tasks of the week.

Each part contains multiple tasks of increasing difficulty.

The tasks of a given week are built on top of each other, therefore you need to attempt them in the order we provide them. Even if you can write a program to solve the last task directly, you still need to write programs for the other tasks in order to pass the tests and be eligible to receive up to full marks for program quality.

The difficult tasks are designed to provide a challenge to most students, therefore it would be perfectly normal and acceptable if you did not complete every single task.

Tips on working in teams

Attend your applied class

Your teammate(s) is in the same applied class. Make sure to attend it to plan your efforts and work together.

Working together

Pair programming

You and your group member/s can sit together and work on a single computer trying to solve each task. This has the benefit of allowing everyone to discuss ideas to solve each part of the problem, as well as double-checking your work as you go along. You'll learn a lot by doing this, and your program will be better.

You can also achieve the same outcome by pair programming online via Zoom or any other collaborative platform.

Work separately

There may be times when you need to work asynchronously. In these cases, you may wish to work directly on your own copy of the code, or you can work in a collaborative way using an Ed workspace to be shared with your group. These allow each member of the team to work on a single copy of the code online (similarly to editing a Google Doc), that way all changes are shared live.

If all teammates are familiar with other tools like git, you are free to use them (however, we will not be able to provide any support if you do).

In any case, all of you will need to copy your code from the ed workspace to the A2 ed lesson to run the automated tests in order to obtain the checkpoint marks.

Resolving issues

If you run into issues (e.g. disagreement, missing teammate), try to resolve them ASAP with your teammate(s).

If that does not work, ask your TA for assistance. Do this as soon as there are issues that you cannot resolve internally.

Tackle team issues as early as they arise. Every semester, we have teams that have unresolved issues by submission day. This generally affects their mark. If you have an issue, do not wait. Ask your TA and document your efforts to resolve the issue, as we will need to know what happened if our arbitrage is needed.

Splitting groups

The assignment is designed to be completed by 2 people, thus, it generally is not an option to complete the assignment on your own. However, in some cases where internal group conflicts cannot be resolved, it may be prudent to split the group so that each individual can continue on with their assignment on their own.

For a group split to occur, you must have collected evidence to show why your group should be split, and you will be required to first raise these issues with your TA. After this, if a resolution cannot be found, you should make a post on Ed directed to the admin team, and they will be able to assist you further.

Valid reasons for splitting a group includes:

- Your teammate leaving the unit, or otherwise not continuing to study,
- They are not communicating with you at all,
- There is any unacceptable behaviour, including bullying or harassment.

Please collect evidence of the above so that it can be shared with the admin team should they require it.



Group splits must occur at least 1 week prior to the deadline. Any requests after this point will be rejected except in extreme circumstances.

Tips on working with automated tests

Handling inputs

You can assume that the inputs provided by the tests conform to the specifications in the brief. This means that you won't need to check that the inputs are correct, whether they are provided by the user or by the automated tests. If you really think there is an error in the tests, then please report it on ed.

The automated tests use complex inputs that may make it hard to debug. We recommend you run your programs with simple inputs that you can manually check.



Make sure that the program you are writing actually expects input or not. If you use the `input()` function when unexpected, you will receive a cryptic `EOF error` from the automarker. If you see this, double check all of your inputs are correct.

Formatting your outputs

In your program, ensure that the format of the output is the same as the format of the examples provided. Every character counts, including spaces and new lines.

Printing new lines

The last character is the newline character `'\n'` in every output file in the automated tests.

```
print("This is on line 1...\nThis is on line two!")
```

The code above produces the output below.

```
This is on line 1...
This is on line two!

```

Notice the that last line is empty. This is because the penultimate line is a `print`, which by default ends with a newline character `'\n'`.

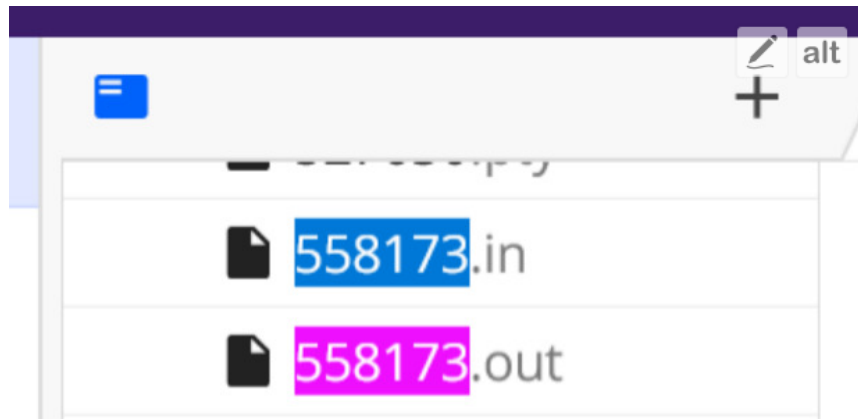
Debugging failed tests

If your program does not pass all the tests, look up the ID of a test that fails:

✓ 558173



Then you can look for the test in your scaffold (including with ctrl+F):



There are two files for each test under the `public` directory of your scaffold:

- `.in` files show the input that is fed to your program.
- `.out` files show the output expected. Beware that some newline characters do not appear because they are in the `.in` file.

Using these files, you can:

- Run the command `python3 name_of_your_program.py < public/558173.in > my_output.out` in the terminal and press Enter to write the output of your program to the file `my_output.out`.
- Run the command `diff my_output.out public/558173.out` in the terminal and press Enter to compare the output of your program to the correct output.

You do not have to do this to complete the assignment, but it may help you on your debugging journey.

Understanding passing test cases

Below is an example of code that passes all 100 **public** tests:

Task 1: Time on Earth (1 mark)		
Description		
Feedback		
1		
Passed		
TESTCASES	100 / 100 passed, 0 points	
558173	✓	0 points
801378	✓	0 points
505717	✓	0 points
811010	✓	0 points

There are two key details to be mindful of. First, is that `100/100 passed` means that this code is passing all 100 of the public tests. If it showed `57/100 passed`, then it would mean only 57 public tests have been passed. Once you pass all the public tests, you will receive a 'green tick' for the whole slide, but this does not necessarily mean that you will pass all private tests and get full marks.



A green tick for the task also has no bearing on the mark you will receive for the quality of the code, as this is assessed manually by the TAs. Make sure that each task conforms to the requirements independently of the number of passing tests.

You will also see the text `0 points` next to every public test and at the top in the summary. This is because public tests are not assessed, hence why they have zero points allocated to them.

Viewing previous Ed submissions

Every time you run the test cases on Ed, it will generate a new 'submission' which we consider for the checkpoint grades. You can see exactly how many test cases you had passed at a particular point in time, as well as review your code for that submission by clicking on it.

Previous **j**

🔍 Maxim Srou



Next **k**

Open Workspace

Submissions

#	SUBMITTED	TESTCASES	RESULT
4	5 days ago	100 / 100	✓
3	5 days ago	73 / 100	✗
2	5 days ago	45 / 100	✗
1	5 days ago	15 / 100	✗

Select a submission



If you have made a change to your code and are now failing more test cases, you can use this to review what you had before or to revert back to it entirely.

How will my work be assessed?

Your assignment will be marked based on 3 criteria:

1. **Program quality**, in terms of simplicity and readability, assessed by a TA who will review your code,
2. **Program correctness**, measured by the output of your programs,
3. **Weekly checkpoints**, to reward timely work.

Automated tests

Both criteria 2 and 3 rely on input/output tests of your programs to measure their correctness. This means we test each of your programs with a series of inputs and measure whether they produce the expected output.

For each task, we put 100 public tests at your disposal to help you create correct programs. They are available both in the scaffold and via the "mark" button at the bottom right of each slide:



However, we will use different, private tests, to evaluate your programs. So, passing a given number of public tests is no guarantee you will achieve a specific mark. Therefore, you can use the public tests to guide you, but, first and foremost, make sure your program follows the assignment brief.

1. Program quality

After you submit on Moodle, a TA will review your code and mark its simplicity and readability. This includes:

1. Clear and meaningful variable names, formatted in snake_case,
2. In-line comments (where required),
3. Documentation (at the top of the file),
4. Code clarity.

The review will occur outside of class, after the Moodle submission deadline.

1.1 Variable names

Variable names should be clear. Following the Python variable naming convention will help you achieve clarity. However, this will not be directly assessed.

Variable names should also be meaningful. This means that the name of the variable should tell a human reader what the role of the variable is in the program, including their type.

```
# Bad variable name
x = int(input("Enter your age: "))

# Good variable name
user_age = int(input("Enter your age: "))
```

The variable names chosen must not override any existing in-built variables. Doing so is bad practice and can lead to difficult-to-debug errors.

```
# This will convert the int 1 into a string '1'
test_conversion = str(1)
print(type(test_conversion))

# This is overriding the existing str() function with a value
# You should NEVER do this
str = "this is a bad idea"

# This code is identical to the code above, but it causes an error because
# the original function has been overridden.
test_conversion = str(1)
print(type(test_conversion))
```

1.2 In-line comments

You are expected to use in-line comments throughout your program to make sure it is clear to the reader. The amount of comments necessary depends on how clear the code is by itself. Therefore, a good habit is to write clear code, so that the amount of in-line comments required is reduced. This will also reduce the number of bugs created.

Comment your code as you write it rather than after reaching a final solution, as it's a lot easier to remember why certain decisions were made and the thought process behind them when the code is fresh in your mind.

Ensure your variable names are meaningful and concise so that your code is understandable at first read. When a reader is going through your program, it should read like a story that is self-explanatory, using in-line comments to explain the logic behind blocks of code and any additional information that is relevant to the functionality of the code.

Avoid repeating yourself by commenting on lines of code that are self-explanatory, and instead reserve your comments to explain higher level logic, such as explaining the overall purpose behind a small block of code.

```
"""
```

```
This is an example of over-commented code.  
Each line is already quite self-explanatory.
```

```
"""
```

```
# This is the first mark  
first_mark = float(input("Enter the first mark: "))  
# This is the second mark  
second_mark = float(input("Enter the second mark: "))  
# Computes the average of the two marks  
average_mark = (first_mark + second_mark) / 2  
# Computes the correctly rounded grade  
rounded_grade = int(average_mark + 0.5)
```

1.3 Documentation

You are expected to include documentation at the start of each file you submit. For example:

```
"""
```

```
This program reads a name from the user and then greets them.
```

```
"""
```

```
name = input("What's your name?\n")  
print("Greetings,", name)
```

You are also expected to write documentation for every function you define.

```
def name_input() -> str:  
    """  
    Prompts the user to input their name, and returns it.  
    """  
  
    name = input("What's your name?\n")  
    return name
```

Documentation should clarify the purpose of a function/file and should provide a high-level overview. Your documentation should not be a rewrite of your comments.



Documentation should be written using docstrings `""" Like this """`, and they should be placed **inside** functions.



If you use comments `# like this`, you won't be getting marks for documentation, as they aren't included in the `__doc__` attribute.

1.4 Code clarity

Optimise your code for readability. In this unit, we do not care about efficiency. So, if the code can be written in multiple ways, opt for the simplest one. A human reader should be able to understand your code quickly and easily.

2. Program correctness

After you submit on Moodle, we will run automated tests on your Moodle submission.

3. Weekly checkpoints

At the end of each week, we expect you to have completed the tasks of the given week.

To measure this, we will look at the percentage of private automated tests passed (i.e. not failed) by your best ed submission made before the weekly checkpoint deadline.

It is therefore necessary for you to run the automated tests on ed if you want to get these marks. You can do so by clicking the "mark" button of each slide.

You are free to edit your code outside ed if you prefer (e.g., a locally installed PyCharm, VSCode, etc), as long as you copy your code on ed to run the tests.



For you to get marks in the checkpoint, you will need to run the automated tests on your assignment in your local Ed workspace. If you don't, you won't be able to get marks, even if your team mate/s do.

Mark breakdown

The assignment has 30 marks, so 1 mark in the assignment equals one mark in the unit.

- Week 6 (Tasks 1-3):
 - Program quality: 4 marks
 - Task 1 correctness: 2 marks
 - Task 2 correctness: 2 marks
 - Task 3 correctness: 1 mark
 - Weekly checkpoint: 1 mark
 - Subtotal: 10 marks
- Week 7 (Tasks 4-6):
 - Program quality: 4 marks
 - Task 4 correctness: 2 marks
 - Task 5 correctness: 2 marks
 - Task 6 correctness: 1 mark
 - Weekly checkpoint: 1 mark
 - Subtotal: 10 marks
- Week 8 (Tasks 7-9):
 - Program quality: 4 marks
 - Task 7 correctness: 2 marks
 - Task 8 correctness: 1.5 marks
 - Task 9 correctness: 1.5 marks
 - Subtotal: 9 marks
- Correctly formatted Moodle submission: 1 mark
- Total: 30 marks

Program quality

Maximum achievable mark

A maximum of 4 marks can be achieved for program quality for each set of 3 tasks.

This is further capped depending on how much of these 3 tasks you have completed.

This is measured by the fraction of the private tests that your programs pass for these 3 tasks.

For instance, if your programs pass 100% of the private tests on task 1, 80% of the tests on task 2 and 30% of the tests on task 3, the quality mark is capped at $(100 + 80 + 30)/300 * 4 = 2.8$ marks.

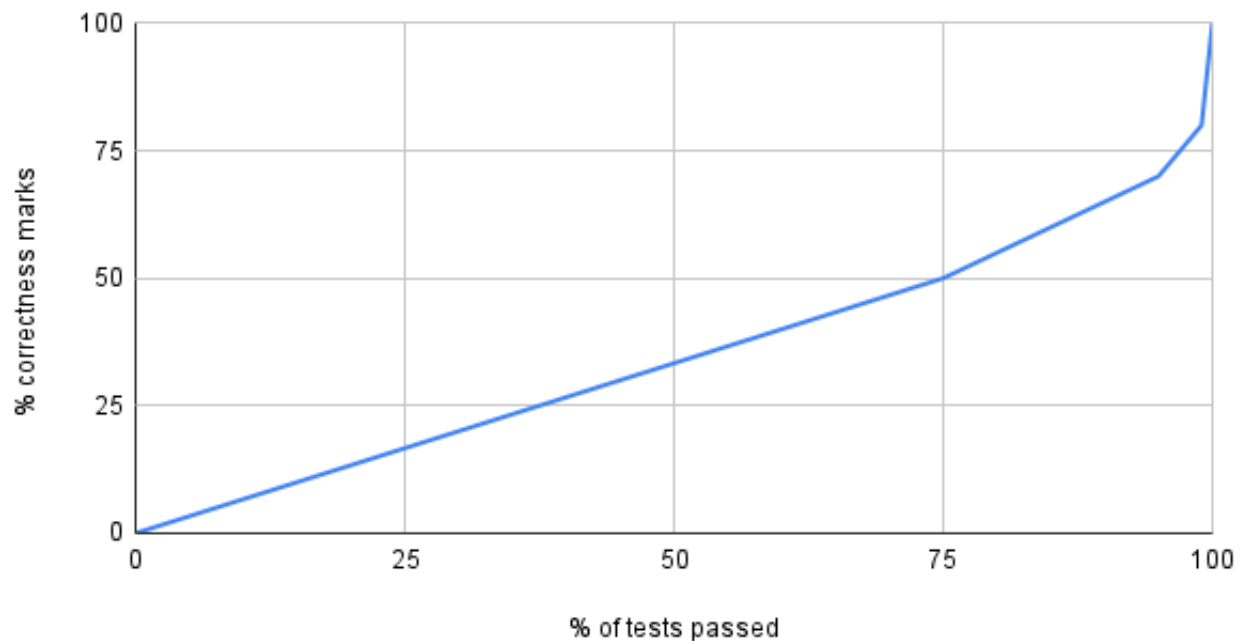
Program correctness

We will evaluate each program with private tests. If your program passes

- 100% of these tests, it receives 100% of the correctness marks,
- 95% to 99% of these tests, it receives 70 to 79% of the correctness marks,
- 75% to 95% of these tests, it receives 50 to 70% of the correctness marks,
- 0% to 75% of these tests, it receives 0 to 50% of the correctness marks.

See chart below.

% correctness marks vs % of tests passed



The rationale is that getting the first 50 tests right is quite easier than getting the last 50 tests right. Furthermore, a program that passes 100% of the tests is more than 1% better than a program that passes 99% of the tests.

Weekly checkpoint

Weekly checkpoint marks are subject to the same curve as program correctness marks.

Penalties

Late penalties

- Up to 7 days: 5% lost per calendar day (or part thereof).
- More than 7 days: Zero (0) marks with no feedback given.

Late penalties are determined using the submission date on Moodle.

Use of external modules

You are allowed to import modules if they are part of the [Python Standard Library](#) (e.g. `math`), but not other modules (e.g. `numpy`). However, note that the solutions do not require the use of modules.

If your solution to a task uses external modules, it will void (i.e. set to 0) any mark it may have received for this task from program correctness, program quality and weekly checkpoint.

Marking rubric

This page is intentionally left blank.

Task 1-3 quality (4 marks)

Write quality code from the get-go.

See how in the slide "How will my work be assessed?".

Task 1: Rabbytes' Age (2 marks)

Fibonacci is having a hard time keeping track of his fast-multiplying rabbytes. He has requested your help to create a database of rabbytes.

Task Description

Write a program that allows a user to create and store new rabbytes. Each rabbit is identified by a unique name. The program also allows the user to enter a rabbit's age.

Main menu

The program prompt of the main menu looks like:

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====
```

If the user inputs 1, 2, 3 or 0, it runs the corresponding function. Otherwise, the prompt is simply printed again. See example 1.



We recommend you create a function for each menu option.

1. Create a Rabbit

Upon inputting **1**, the program will then print the prompt

```
Input the new rabbit's name:
```

If the user enters a new name, the program returns to the menu and prints the main prompt.

Otherwise, the program prints

```
That name is already in the database.
Input the new rabbit's name:
```

and repeat this until the user inputs a new name. See example 2.

2. Input Age of a Rabbit

Upon inputting `2`, the program will then print the prompt

```
Input the rabbit's name:
```

If the user inputs a name (e.g. `rabbie`) that is already in the database, the program asks for the rabbit's age:

```
Input the rabbit's name:
```

```
rabbie
```

```
Input rabbie's age:
```

```
2
```

It then returns to the main menu and prints the main prompt.

If the user inputs a name that does not exist, the program prints

```
That name is not in the database.
```

```
Input the rabbit's name:
```

until an existing name is provided. See example 3.



We recommend you store the age of a rabbit using a dictionary. See [pre-class content](#).



We assume that:

- the age input is an integer,
- the user will only pick option 2 if a rabbit has already been created,
- the user can freely set the age of a rabbit, even if it already has one. It will be updated.

3. List Rabbytes

Upon inputting `3`, the program will then print

```
Rabbytes:
```

```
rabbie (42)
```

```
not_rabbie (Unknown)
```

if two rabbytes, `rabbie` and `not_rabbie`, have been created before, and if `rabbie`'s age (42) has been input, and `not_rabbie`'s age hasn't. See example 4.



Display the rabbytes in the order they were created.

Examples

User inputs are in **bold font** below.

Example 1

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====
foo
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====
0
```

Example 2

```

=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====
1
Input the new rabbit's name:
undercover_robbie
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====
1
Input the new rabbit's name:
undercover_robbie
That name is already in the database.
Input the new rabbit's name:
robbie_the_rabbit
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====
0

```

Example 3

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====

1
Input the new rabbit's name:
rabbie
=====

Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====

2
Input the rabbit's name:
bob
That name is not in the database.
Input the rabbit's name:
rabbie
Input rabbie's age:
2
=====

Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====

0
```

Example 4

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.
=====

3
Rabbytes:
```


=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.

=====

1

Input the new rabbit's name:

rabbie

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.

=====

1

Input the new rabbit's name:

not_rabbie

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.

=====

2

Input the rabbit's name:

rabbie

Input rabbie's age:

42

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
0. Quit.

=====

3

Rabbytes:

rabbie (42)

not_rabbie (Unknown)

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.

0. Quit.
=====

0

Task 2: Rabbytes' Lineage (2 marks)

Impressed with your work, Fibonacci wants you to improve your program to track the lineage of his wacky rabbytes.

Task Description

Write a program that, on top of the previous options, allows the user to specify a parent/kitten relationship.

Main menu

The main prompt now has an extra two options:

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.
=====
```



We recommend you create a function for each menu option, and reuse functions you have written (by improving them if need be) if possible.

4. Create a Parental Relationship

This menu allows the user to indicate a parental relationship between two rabbytes, by first inputting the name of the parent, then the name of the kitten.

```
Input the parent's name:
Anakin
Input the kitten's name:
Luke
```

If either name does not correspond to an existing rabbit, a rabbit with that name is created, as it would be if that name was provided through option 1 of the menu. See example 1.



You do not need to verify any consistency of the input. In particular, you may suppose that the user and the automated tests will:

- ensure that the number of parents is between 0 and 2.
 - a rabbit cannot be their own ancestor.
- However, make no other assumptions as to how rabbytes reproduce.

5. List Direct Family of a Rabbit

This menu asks the user to enter a name, and then displays its parents and kittens.

Input the rabbit's name:

Anakin

Parents of Anakin:

Kittens of Anakin:

Leia

Luke

If the input name is not a known rabbit, the program prints

That name is not in the database.

Input the rabbit's name:

until a known name is provided. The parents and kittens are then displayed as above. See example 2.



Assume that the user (and the automated tests) will only pick option 5 if a rabbit has already been created.



Display the names of the parents and the kittens in alphabetical order. You may use [Python's built-in sorting utilities](#).

Examples

User inputs are in **bold font** below.

Example 1

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.
=====
4
Input the parent's name:
```

Anakin

Input the kitten's name:

Luke

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.

=====

1

Input the new rabbit's name:

Leia

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.

=====

4

Input the parent's name:

Anakin

Input the kitten's name:

Leia

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.

=====

3

Rabbytes:

Anakin (Unknown)

Luke (Unknown)

Leia (Unknown)

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.

```
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.
```

```
=====
```

```
5
```

```
Input the rabbit's name:
```

```
Anakin
```

```
Parents of Anakin:
```

```
Kittens of Anakin:
```

```
Leia
```

```
Luke
```

```
=====
```

```
Enter your choice:
```

```
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.
```

```
=====
```

```
5
```

```
Input the rabbit's name:
```

```
Leia
```

```
Parents of Leia:
```

```
Anakin
```

```
Kittens of Leia:
```

```
=====
```

```
Enter your choice:
```

```
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.
```

```
=====
```

```
0
```

Example 2

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.

=====

4

Input the parent's name:

Anakin

Input the kitten's name:

Leia

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.

=====

5

Input the rabbit's name:

Luke

That name is not in the database.

Input the rabbit's name:

Leia

Parents of Leia:

Anakin

Kittens of Leia:

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
0. Quit.

=====

0

Task 3: Rabbytes' Extended Universe (1 mark)

Just when you thought your program met Fibonacci's expectations, he barges into your room, out of breath, and screams "they are out of control"! He explains that they are nearing their critical mass (a terabyte), and that the impending catastrophe can only be avoided if your program helps him isolate rabbit cousins.

Task Description

Main menu

The main prompt now has an extra option:

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
6. Find Cousins of a Rabbit.
0. Quit.
=====
```



We recommend you create a function for each menu option, and reuse functions you have written (by improving them if need be) if possible.

6. Find Cousins of a Rabbit

This menu allows the user to list the cousins of a rabbit. A cousin is a kitten of a sibling of a parent.

```
Input the rabbit's name:
cousin1
Cousins of cousin1:
cousin2
```

Similar to menu option 5, it asks the user for a name again

```
That name is not in the database.
Input the rabbit's name:
```


until it corresponds to one in the database. See example 1.



Display the names of the cousins in alphabetical order. You may use [Python's built-in sorting utilities](#).



A rabbit cannot be its own cousin. Every other rabbit can.

Examples

User inputs are in **bold font** below.

Example 1

```
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
6. Find Cousins of a Rabbit.
0. Quit.
=====
4
Input the parent's name:
grandma
Input the kitten's name:
dad1
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
6. Find Cousins of a Rabbit.
0. Quit.
=====
4
Input the parent's name:
grandma
Input the kitten's name:
dad2
=====
Enter your choice:
1. Create a Rabbit.
2. Input Age of a Rabbit.
```

3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
6. Find Cousins of a Rabbit.
0. Quit.

=====

4

Input the parent's name:

dad1

Input the kitten's name:

cousin1

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
6. Find Cousins of a Rabbit.
0. Quit.

=====

4

Input the parent's name:

dad2

Input the kitten's name:

cousin2

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.
6. Find Cousins of a Rabbit.
0. Quit.

=====

6

Input the rabbit's name:

cousin1

Cousins of cousin1:

cousin2

=====

Enter your choice:

1. Create a Rabbit.
2. Input Age of a Rabbit.
3. List Rabbytes.
4. Create a Parental Relationship.
5. List Direct Family of a Rabbit.

6. Find Cousins of a Rabbit.

0. Quit.

=====

0

Week 6 checkpoint (1 mark)

The tasks of week 6 are Tasks 1, 2 and 3.

The checkpoint is Friday, August 30, 2024, 23:55 (Melbourne time), 21:55 (Malaysia time).

This page is intentionally left blank.

Task 4-6 quality (4 marks)

Write quality code from the get-go.

See how in the slide "How will my work be assessed?".

Task 4: List variables (2 marks)

Fibonacci seems exhausted. Intrigued, I mean concerned, you ask Fibo what's on his mind. "I've been marking assignments all weekend", he confesses, while yawning. "What drives me mad is the inconsistency of the variable formatting." You wouldn't happen to be able to help my students with that, would you?". Turns out, you are actually working on the perfect tool to help your mate Fibo!

Task Description

Write a program that allows a user to list the variables of a Python program. We suppose that each variable follows one of three cases:

- `snake_case`,
- `camelCase`, or
- `PascalCase` (a.k.a. Upper camel case).



The programs provided as input may not make sense or be correct. Ignore that.

Program input

When the program is run, the user first inputs the program. For example:

```
Enter the Python program to analyze, line by line. Enter 'end' to finish.  
a = b + c  
end
```

Main menu

Once the program is input, the main menu is displayed. The program prompt of the main menu looks like:

```
=====  
Enter your choice:  
1. Print program.  
2. List.  
0. Quit.  
=====
```

If the user inputs 1, 2 or 0, it runs the corresponding function. Assume no other input is provided.



We recommend you create at least one function for each menu option.

1. Print program

Upon inputting **1**, your program will then print the program that was provided in input (without the "end"):

```
=====
Enter your choice:
1. Print program.
2. List.
0. Quit.
=====
1
Program:
a = b + c
```



The programs provided as input may have indentation and/or extra spaces. They need to appear as provided when printing the program. See example 1.

2. List

Upon inputting **2**, your program will list the variables of the input program. They are provided in alphanumerical order. Each variable is listed only once.

```
=====
Enter your choice:
1. Print program.
2. List.
0. Quit.
=====
2
Variables:
a
b
c
```



Use Python's sorting functions, such as `sorted`. You will notice that uppercase words will be listed before lowercase words. This is what we want.

```
print(sorted(["robbie", "Robbie"]))
```



- For the purpose of this and the following task, all variable names
1. start with a lowercase or uppercase character, i.e. `a-z` and `A-Z`,
 2. may only contain lowercase or uppercase character, i.e. `a-z` and `A-Z`, and underscores, i.e. `_`,
 3. cannot be a Python keyword,

4. are surrounded by at least one space on either side.



The programs provided may contain Python keywords. These are not variables. You can programmatically obtain a list of the possible keywords using `keyword.kwlist`. See example below.

```
import keyword
print(keyword.kwlist)
```



The input programs do not contain string literals, such as "hello".

Examples

User inputs are in **bold font** below.

Example 1

Enter the Python program to analyze, line by line. Enter 'end' to finish.

```
myVar =  your_var + TheirVar
for your_var in newVar :
    myVar  += TheirVar
end
```

=====

Enter your choice:

- 1. Print program.
- 2. List.
- 0. Quit.

=====

2

Variables:

TheirVar
myVar
newVar
your_var

=====

Enter your choice:

- 1. Print program.
- 2. List.
- 0. Quit.

=====

1

Program:

```
myVar =  your_var + TheirVar
for your_var in newVar :
    myVar  += TheirVar
```

=====

Enter your choice:

- 1. Print program.
- 2. List.
- 0. Quit.

=====

0

Task 5: Formatting variables (2 marks)

Task Description

Write a program that, on top of the previous options, offers to format variables to snake_case.

Main menu

The main menu now looks like:

```
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
0. Quit.
=====
```

3. Format

Upon inputting **3**, the program asks you to pick one of the variables, and changes its case to snake_case throughout the whole program. For example, if a variable name is:

1. `RobbieTheRobbot` or `robbieTheRobbot`, then it becomes `robbie_the_robbot`,
2. `robbie_the_robbot`, then it remains `robbie_the_robbot`.

If the variable name input by the user does not exist, the program asks again. This is case-sensitive.

Examples

User inputs are in **bold font** below.

Example 1

```
Enter the Python program to analyze, line by line. Enter 'end' to finish.
aBC = DeF + a_b_x
end
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
```

0. Quit.

=====

2

Variables:

DeF

aBC

a_b_x

=====

Enter your choice:

1. Print program.

2. List.

3. Format.

0. Quit.

=====

3

Pick a variable:

aBC

=====

Enter your choice:

1. Print program.

2. List.

3. Format.

0. Quit.

=====

1

Program:

a_b_c = DeF + a_b_x

=====

Enter your choice:

1. Print program.

2. List.

3. Format.

0. Quit.

=====

3

Pick a variable:

skibidi

This is not a variable name.

Pick a variable:

a_b_x

=====

Enter your choice:

1. Print program.

2. List.

3. Format.

0. Quit.

=====

2

Variables:

DeF

a_b_c

a_b_x

=====

Enter your choice:

1. Print program.

2. List.

3. Format.

0. Quit.

=====

0



Assume no clash is possible, i.e., in the same program, there cannot be both a variable named `RobbieTheRobbot` and `robbie_the_robbot`.

Task 6: Renaming variables (1 mark)

Task Description

Write a program that, on top of the previous options, offers to rename variables.

Main menu

The main menu now looks like:

```
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
```

4. Rename

Upon inputting **4**, the program asks you to pick one of the variables, then a new name, and then changes the old name to the new name throughout the whole program.

If the variable name input by the user does not exist, the program asks again. If the new name is already in use, then the program asks again. This is case-sensitive.



Assume that the new names follow either snake_case, camelCase or PascalCase.

Examples

User inputs are in **bold font** below.

Example 1

```
Enter the Python program to analyze, line by line. Enter 'end' to finish.
aBC = DeF + a_b_x
DeF -= aBC + test
end
=====
Enter your choice:
```

```
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
4
Pick a variable:
a_b_x
Pick a new variable name:
test
This is already a variable name.
Pick a new variable name:
newVar
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
1
Program:
aBC = DeF + newVar
DeF -= aBC + test
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
3
Pick a variable:
newVar
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
2
Variables:
DeF
```

```
aBC
new_var
test
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
1
Program:
aBC = DeF + new_var
DeF -= aBC + test
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
4
Pick a variable:
aBC
Pick a new variable name:
meaningful_name
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
1
Program:
meaningful_name = DeF + new_var
DeF -= meaningful_name + test
=====
Enter your choice:
1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.
=====
```


Example 2

Enter the Python program to analyze, line by line. Enter 'end' to finish.

robbie = the_goat

end

=====

Enter your choice:

1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.

=====

4

Pick a variable:

fibo

This is not a variable name.

Pick a variable:

robbie

Pick a new variable name:

still_robbie

=====

Enter your choice:

1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.

=====

2

Variables:

still_robbie

the_goat

=====

Enter your choice:

1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.

=====

1

Program:

still_robbie = the_goat

=====

Enter your choice:

1. Print program.
2. List.
3. Format.
4. Rename.
0. Quit.

=====

0

Week 7 checkpoint (1 mark)

The tasks of week 7 are Tasks 4, 5 and 6.

The checkpoint is Friday, September 6, 2024, 23:55 (Melbourne time), 21:55 (Malaysia time).

This page is intentionally left blank.

Task 7-9 quality (4 marks)

Write quality code from the get-go.

See how in the slide "How will my work be assessed?".

Task 7: Table Duplication (2 marks)

Task Description

Write a program that allows a user to display and duplicate tables. At the beginning of the program, some tables have to be read from csv files that you must not modify.

 Use the module `tabulate` to print tables in the expected format.

 Do not write into files for any of the tasks.

Main menu

The program prompt of the main menu looks like:

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.
=====
```

If the user inputs 1, 2, 3 or 0, it runs the corresponding menu option.

 Assume that the user and the automated tests input a correct choice in the main menu.

 We recommend you create a function for each menu option.

1. List tables

This menu option prints a table that lists the tables that currently exist in the program.

Index	Columns	Rows
0	5	6
1	4	7
2	3	4
3	3	6

After the header, there is one row per table, with its index, and the number of columns and rows of

that table.

At the beginning of the program, all four tables are shown in the list in the order:

1. grades.csv,
2. class_students.csv,
3. rabbytes_club_students.csv,
4. rabbytes_data.csv,

which then looks exactly as shown above. This list is updated as tables change. See example 1.

2. Display table

This menu option prints a table given its index.

The program prints `Choose a table index (to display):`. The user then selects a table by its index (as listed by menu option 1). See below.

Choose a table index (to display):

1

Student ID	First Name	Last Name	Grade Code
-----	-----	-----	-----
798154	Brynhildr	Blakeley	N
134789	Felix	Li	N
798951	Paityn	Summers	P
465120	Turnus	Elliot	C
963245	Alysia	Jervis	D
469120	Muhammad	Saad	HD

If an incorrect table index is provided, the program prints `Incorrect table index. Try again.`, and prints `Choose a table index (to display):` again, until a correct index is provided. See example 2.



For future menu options (including for Tasks 8 and 9), you will also need to check that the table index is correct, and ask again if need be.



For menu option 2 and future menu options, assume that, when asked to input a table index, the user inputs an integer.

3. Duplicate table

Similar to menu option 2, this menu option repeatedly asks the user to select a table, then creates a copy of that table. The index of the new table is the smallest index that has not previously been assigned to a table.

Choose a table index (to duplicate):
1

See example 3.

Examples

User inputs are in **bold font** below.

Example 1

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.
=====
1
  Index      Columns      Rows
  -----
      0         5         6
      1         4         7
      2         3         4
      3         3         6
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.
=====
0
```

Example 2


```

=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.
=====
2
Choose a table index (to display):
4
Incorrect table index. Try again.
Choose a table index (to display):
3
Rabbit      Birth year  Favorite treat
-----
Rabbit_1      2022    Carrots
Rabbit_2      2023    Celery
Rabbit_3      2023    Broccoli
Rabbit_4      2024    Cabbage
Rabbit_5      2022    Lettuce
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.
=====
0

```

Example 3

```

=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.
=====
3
Choose a table index (to duplicate):
1
=====
Enter your choice:
1. List tables.
2. Display table.

```

3. Duplicate table.
0. Quit.

=====

1

Index	Columns	Rows
-----	-----	-----
0	5	6
1	4	7
2	3	4
3	3	6
4	4	7

=====

Enter your choice:

1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.

=====

2

Choose a table index (to display):

4

Student ID	First Name	Last Name	Grade Code
-----	-----	-----	-----
798154	Brynhildr	Blakeley	N
134789	Felix	Li	N
798951	Paityn	Summers	P
465120	Turnus	Elliot	C
963245	Alysia	Jervis	D
469120	Muhammad	Saad	HD

=====

Enter your choice:

1. List tables.
2. Display table.
3. Duplicate table.
0. Quit.

=====

0

Task 8: Table Creation and Deletion (1.5 marks)

Task Description

Write a program that, on top of the previous options, allows a user to create and delete a table.

Main menu

The program prompt of the main menu looks like:

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
0. Quit.
=====
```

If the user inputs 1, 2, 3, 4, 5, or 0, it runs the corresponding menu option.

4. Create table

This menu creates a table from an existing table by selecting some of its columns in a certain order.

It repeatedly asks the user `Choose a table index (to create from):` until a correct index is input.

It then asks the user `Enter the comma-separated indices of the columns to keep: .`

Each index must be between 0 and the current number of columns of that table, minus one.

The order of the indices determines the order of the columns of the created table.

The index of the new table is the smallest index that has not previously been assigned to a table.

See example 1.



Assume that the user and test inputs are well-formatted (e.g. `3,1`, without spaces) and the set of indices is valid.

5. Delete table

This menu option repeatedly asks the user to select a table by its index (as shown by menu option 1) with `Choose a table index (for table deletion):`, then deletes the table.

Note that deleting a table does not change the indices of other tables. See example 2.

Examples

User inputs are in **bold font** below.

Example 1

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
0. Quit.
=====
4
Choose a table index (to create from):
6
Incorrect table index. Try again.
Choose a table index (to create from):
1
Enter the comma-separated indices of the columns to keep:
3,1
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
0. Quit.
=====
2
Choose a table index (to display):
4
Grade Code      First Name
-----
N                Brynhildr
N                Felix
P                Paityn
C                Turnus
D                Alysia
```

```

HD                Muhammad
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
0. Quit.
=====
0

```

Example 2

```

=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
0. Quit.
=====
5
Choose a table index (for table deletion):
2
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
0. Quit.
=====
1

```

Index	Columns	Rows
0	5	6
1	4	7
3	3	6

```

=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.

```

4. Create table.

5. Delete table.

0. Quit.

=====

2

Choose a table index (to display):

2

Incorrect table index. Try again.

Choose a table index (to display):

1

Student ID	First Name	Last Name	Grade Code
-----	-----	-----	-----
798154	Brynhildr	Blakeley	N
134789	Felix	Li	N
798951	Paityn	Summers	P
465120	Turnus	Elliot	C
963245	Alysia	Jervis	D
469120	Muhammad	Saad	HD

=====

Enter your choice:

1. List tables.

2. Display table.

3. Duplicate table.

4. Create table.

5. Delete table.

0. Quit.

=====

5

Choose a table index (for table deletion):

2

Incorrect table index. Try again.

Choose a table index (for table deletion):

3

=====

Enter your choice:

1. List tables.

2. Display table.

3. Duplicate table.

4. Create table.

5. Delete table.

0. Quit.

=====

0

Task 9: Column Deletion and Table Restoration (1.5 marks)

Task Description

Write a program that, on top of the previous options, allows a user to delete a column and restore a table.

Main menu

The program prompt of the main menu looks like:

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
```

If the user inputs 1, 2, 3, 4, 5, 6, 7 or 0, it runs the corresponding menu option.

6. Delete column

This menu option allows the user to delete a column from a table.

Similar to other menu options, the program repeatedly prints `Choose a table index (for column deletion):` until a correct index is input.

Then the program prints `Enter the index of the column to delete: .`

The user then inputs one column index, between 0 and the current number of columns of that table, minus one.

That column is then deleted, which is for example reflected in menu option 1 and 2.

See example 1.



Assume that the column index is valid.



Deleting a column in a table does not affect any other table.

7. Restore table

This menu option allows a user to restore a table that has been deleted via menu option 5.

It prompts `Choose a table index (for restoration):` and reads a table index from the user.

If the table index is invalid (either the table exists and has not been deleted, or has never existed), then the program prints `Incorrect table index. Try again.`, and iterates until a valid index has been input.

The deleted table is then restored with the index it had prior to deletion.



Assume that this menu option is only used if a table has been deleted.

Examples

User inputs are in **bold font** below.

Example 1

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
1
  Index    Columns    Rows
  -----
      0         5       6
      1         4       7
      2         3       4
      3         3       6
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
```



```

5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
6
Choose a table index (for column deletion):
6
Incorrect table index. Try again.
Choose a table index (for column deletion):
2
Enter the index of the column to delete:
1
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
1
  Index      Columns      Rows
  - - - - -  - - - - -  - - - - -
      0           5           6
      1           4           7
      2           2           4
      3           3           6
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
2
Choose a table index (to display):
2
First Name      Rabbit
- - - - -      - - - - -
Brynhildr       Rabbit_1

```

```
Turnus          Rabbit_2
Jamaluddin      Rabbit_5
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
0
```

Example 2

```
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
5
Choose a table index (for table deletion):
2
=====
Enter your choice:
1. List tables.
2. Display table.
3. Duplicate table.
4. Create table.
5. Delete table.
6. Delete column.
7. Restore table.
0. Quit.
=====
7
Choose a table index (for restoration):
1
Incorrect table index. Try again.
```

Choose a table index (for restoration):

2

=====

Enter your choice:

- 1. List tables.
- 2. Display table.
- 3. Duplicate table.
- 4. Create table.
- 5. Delete table.
- 6. Delete column.
- 7. Restore table.
- 0. Quit.

=====

1

Index	Columns	Rows
0	5	6
1	4	7
2	3	4
3	3	6

=====

Enter your choice:

- 1. List tables.
- 2. Display table.
- 3. Duplicate table.
- 4. Create table.
- 5. Delete table.
- 6. Delete column.
- 7. Restore table.
- 0. Quit.

=====

0

Final submission on Moodle (1 mark)



Only one teammate per team needs to submit on Moodle, on behalf of the whole team.

Your final submission must be one zip file with the necessary and sufficient files, as shown below.

```
A2_<student_1_id>_<student_2_id>.zip
```

```
└─> family_database_age.py
└─> family_database_cousins.py
└─> family_database_parents.py
└─> table_menu_create_delete.py
└─> table_menu_delete_restore.py
└─> table_menu_duplicate.py
└─> variable_renaming_format.py
└─> variable_renaming_list.py
└─> variable_renaming_rename.py
```

Alternatively, you may also submit in the following format.

```
A2_<student_1_id>_<student_2_id>.zip
```

```
└─> A2_<student_1_id>_<student_2_id>
    └─> family_database_age.py
    └─> family_database_cousins.py
    └─> family_database_parents.py
    └─> table_menu_create_delete.py
    └─> table_menu_delete_restore.py
    └─> table_menu_duplicate.py
    └─> variable_renaming_format.py
    └─> variable_renaming_list.py
    └─> variable_renaming_rename.py
```

If your submission follows the structures above, you will receive one mark. Make sure that you replace `<student_i_id>` with your actual student ID, and ensure that you use the correct names for each of the files, including the file extension.

For more information, please check the [Ed post](#).



If you are by yourself in your team, use `A2_<student_id>` for the file and directory names.

If you are in a team of 3, use A2_<student_1_id>_<student_2_id>_<student_3_id>



Some operating systems hide the file extension, so be sure to check if that's the case before "adding it back in".

This page is intentionally left blank.

Changelog (9 changes)

Change 9 - 10/09

Added the marking rubric.

Change 8 - 05/09

Added the following info bubble to Task 7:

 For future menu options (including for Tasks 8 and 9), you will also need to check that the table index is correct, and ask again if need be.

Change 7 - 03/09

Added the following info bubble to Task 4:

 The input programs do not contain string literals, such as "hello".

Change 6 - 02/09

In the example 1 of Task 4, removed erroneous bold font:

```
Enter the Python program to analyze, line by line. Enter 'end' to finish.
myVar =  your_var + TheirVar
for your_var in newVar :
    myVar  += TheirVar
end
=====
Enter your choice:
1. Print program.
2. List.
0. Quit.
=====
2
Variables:
TheirVar
myVar
newVar
your_var
=====
```

Thanks Li for [pointing this out!](#)

Change 5 - 29/08

Added the following info bubble to Task 1:

 We recommend you store the age of a rabbit using a dictionary. See [pre-class content](#).

Change 4 - 29/08

Added example 2 to Task 6.

Change 3 - 28/08

Updated the assignment settings so that it no longer shows a "Start Attempt" button and a "Submit" button. It is now identical to how A1 was conducted in that you just press the "Test" button to run the test cases against your code.

Change 2 - 27/08

In Task 4, added a fourth bullet point to this list:



For the purpose of this and the following task, all variable names

1. start with a lowercase or uppercase character,i.e. `a-z` and `A-Z`,
2. may only contain lowercase or uppercase character,i.e. `a-z` and `A-Z`, and underscores, i.e. `_`,
3. cannot be a Python keyword,
4. are surrounded by at least one space on either side.

Thanks to Yuvraj for [asking](#)!

Change 1 - 26/08

Edits to the name of the files required for the Moodle upload.